Developer's Document

The code has been written in C++ and it is an objected-oriented program. Code files have the extension .cpp or .h and all located in "`Solver`" folder. The program includes three code files which described as follow:

1. NUMERICALDERIVATION.h

This file defines a class which initializes $u(x,t)$; t =0. Also, it calculates the first and second numerical derivatives of field $u(x,t)$ by x. Finite difference method is used to approximate the derivatives of field $u(x,t)$. The number of grid points and grid size are set in this class as well.

1.1 Class:

Class name: `mygrid`

Description: includes the values of $u(x,t)$ on the grid points and calculates their first and second derivation.

Class members:

1. `mygrid()`:constructor (Initializes the variables 'u'). This function has a default initializer, unless the user uses ones input file.
2. `xDim`: number of grids (this is the same as variable "`Dim`")
3. `u[xDim]`: this array represent field $u(x,t)$
4. `Delt_x`: dx
5. `first_dev(int i)`: function that calculates the first derivative
6. `sec_dev(int i)`: function that calculates the second derivative

2. Filter.cpp

This function filters the result obtained by DNS solution. Therefore, it can be compared with LES solution data with the same number of grids. For instance, the algorithm of filtration for point with index I, (x_i), and filter size = 2* dx can be written as:

$\bar{u}\ (x_i) = (u\ (x_{i-2}) + u\ (x_{i-1}) + u(x_i) + u(x_{i+1}) + u(x_{i+2}))/5\ ;\ t = t_0$

This algorithm assumes that the boundaries are periodic.

2.1. Function:

Name: Filter

Variables:

1. h: a pointer which gives access to the components of variable '`u`'.

2. l: a pointer which gives access to the components of variable '`u_bar`'. (This variable represent the filtered field $\bar{u}\ (x_i)$)
3. `size`: filter size

3. MainProgram.cpp

The one dimensional Burgers' equation is solved numerically. The form of the Burgers equation considered here is:

$$\frac{du}{dt} + u\frac{du}{dx} = \vartheta\frac{d^2u}{dx^2}\ ;\ \ 0 \le x \le 1\ \ and\ t \ge 0 \qquad (1)$$

The numerical method used is finite difference method (second order in space and first order in time). Two types of solution for Burgers' equation were mentioned here:
1. DNS (direct numerical solution)
2. LES (large eddy simulation).

In DNS, the finite difference scheme of Burgers' equation is solved in high resolution to capture all information. In large eddy simulation (LES), an artificial viscosity is added to normal viscosity (ϑ +LESvis). So, the resolution can be reduced and save us computational cost.

3.1. Variables

`Kinvis`: viscosity
`It`: number of iterations
`chk, k`: chk=number of time steps to write the output file, k= output file index
`a[Dim]`: intermediate variable which is similar to '`u`' in NUMERICALDERIVATION.h
`u_bar[Dim]`: filtered velocity
`filter_opt`: a Boolean variable to active the filtration

`size`: filter size
`Cs`: Smagorinsky model coefficient
`vis_LES [Dim]`: an array represent the numerical viscosity for LES approach
`LES_opt`: A Boolean variable to active the LES option

3.2 Functions

`Outputfile`: This function writes the solution in an output file. This function receive the filter size to reduce the size of DNS data to the size of LES results.